

Grid Rearrangement

1. What we claim

The problem. A policy rearranges rigid multi-cell pieces (polyominoes of 1–8 cells) on a grid into a target colour pattern: one move teleports a piece to any free spot, rotated by 90° if useful; same-colour pieces are interchangeable; boards are dense (55–80 % covered); cost is the distance moved.

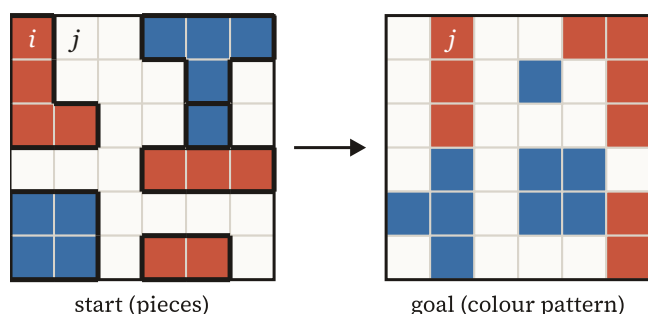
The claim. A policy can be told about the board in two ways. It can be given **names**: this cell is colour 3, this cell belongs to piece 7, this piece is shape 412, this cell sits in row 5. Or it can be given **relations between pairs of cells**: are these two cells the same colour? part of the same piece? the same shape type? how far apart? A name means something only if the network met it in training — colour 9, an 80th piece, a shape that never occurred, row 17 mean nothing to it. A relation is the same question on any board. We train the same policy both ways, from scratch by RL on 12×12 boards, changing one input at a time and giving the name version at least as much information. Wherever a test goes past what the names covered in training — more colours, a never-seen shape, more pieces on one board, a bigger board — the relational policy keeps solving for less cost and, unlike the name version, still works with a single greedy rollout; where the names were covered, the two are close. Relations cost nothing inside the training range, and they are what carries the policy outside it.

What it is not. Not “RL beats planning”: a purpose-built A* solves these boards within small search budgets. The domain is a controlled testbed in which one representation choice can be isolated cleanly.

2. Where it stands in the literature

- **Learned rearrangement and packing** (Scene Mover, TAP-Net, Transporter / CLIPort, container pre-marshalling; 2020–21): rotation, blocking or interchangeable goals each appear, never multi-cell pieces + rotation + a colour-pattern goal + dense packing, and no representation is compared. Unseen-shape transfer exists in packing (Zhao et al. 2023: ~20 % held-out shapes; Transporter: via a vision encoder) — not with unseen shapes dominating the board, a matched control and a mirror-shape check.
- **Symbol-invariant transformers** (“Names Don’t Matter”, Symbol-Equivariant Recurrent Reasoning Models — ICML 2026; Renamer 2023): renaming invariance for supervised models via one slot per symbol or a hard attention mask; **structure-biased attention in policies** (SWAT 2022, Body Transformer 2024): a fixed graph over whole-object tokens. *Gap*: relations as soft attention terms in every layer of an acting policy, binding cells into objects without object tokens.
- **Relative vs absolute position** (Joshi et al. 2021: policies that decode well greedily transfer badly; INVit / ELG 2024; “Relative positions generalize, absolute positions memorize”, 2026). *Gap*: a controlled comparison inside a spatial RL policy in which the trade-off disappears.

3. Setup, and the one idea



What the network is told about a pair of cells (i, j)

- same colour now? — same colour in the goal? — i 's colour now = j 's goal colour?
- same piece now / in the goal? — same shape type (up to rotation)? — can i 's piece fill the goal slot at j ?
- offset $i \rightarrow j$: Δrow , Δcol , distance

Marked pair: different pieces; j is empty now; i 's colour = j 's goal colour; offset (0, +1). Not: “ i is red”, “ i is at row 0”.

Start: pieces (thick outlines) in two colours. Goal: a colour pattern only — any piece of the right colour may cover any goal cell, rotated as needed. ☐ ☐

Setup. Training boards up to 12×12 (larger only at test); pieces of 1–8 cells (up to ~35 per board, ~80 at 20×20); 2–4 colours in training. A move picks a piece, a rotation and a free position; the episode ends when the pattern matches or after 2.5 moves per piece; cost = mean distance the piece's cells travel. Policy, shared by all variants: one token per cell in a small transformer, a pointer head picks the piece and then its pose; REINFORCE from scratch, curriculum up to 12×12, 3 seeds per variant.

The one idea. For every pair of cells the network receives the answers in the figure, and each answer adds a learned amount to the attention score between the two cells, in every layer — the “addition” from our meeting; what is added is the relation. Per cell it gets only *occupied* / *goal-occupied* / *already matching* and a few numbers

about the covering piece (size, extent, symmetry). No colour value, piece identity, shape name or coordinate is ever an input. Two things follow: invariance to renaming, to more colours, to new shapes and to bigger boards is built in rather than approximated by augmentation; and the quantities the task actually turns on — which cells match their goal, which move together, which piece fills which slot — are handed over instead of being learned from names. Robotics analogue: relative transforms between objects instead of world-frame coordinates, here applied to labels and shapes as well.

4. Results

Reading the tables. Each policy is run three ways on every test board. *greedy*: at each step take the policy’s single most likely move — the cheapest way to use a policy (one rollout, no search). *×1*: one rollout with moves sampled from the policy. *×16*: sixteen sampled rollouts, keep the best — a 16× search budget. *Solve* = fraction of test boards solved within the step budget (500 boards per setting, 300 at 20×20; all solvable): greedy solve shows whether the policy commits to good moves on its own, ×16 whether it can solve at all given search. *cost* = move distance of the best rollout, on the boards that all compared policies solved (so it shifts slightly between tables). ↑ higher is better, ↓ lower is better. a / b / c = the three seeds; otherwise seed means. Relational vs symbol is always 3 seeds vs 3 seeds; *separated* = every seed of one lies beyond every seed of the other.

The content claim at a glance

Relational policy vs the control that receives the same content as a per-cell code. cost ↓ = how much cheaper the relational policy is; greedy ↑ = its greedy solve-rate advantage. Means over the test settings of a board size; an arrow reads smaller board → larger board. Seeds separated unless marked; 12×12 unless stated.

relation	symbol control	within the training exposure	beyond it
same colour?	one-hot colour, with or without relabelling	4 colours, as trained: cost −11 %, <i>seeds overlap</i> ; greedy +27 pp	6–10 colours: cost −17 % → −42 %, greedy +42 → +51 pp (12×12 → 20×20)
same shape type?	shape ID (one of 989 shapes)	training shapes: cost −29 %; greedy +44 pp	cost −38 % → −42 %, greedy +54 → +51 pp (16×16 → 20×20). Never-seen shapes: relational unchanged, the ID control loses a further 5–10 pp
same piece?	piece ID, drawn at random per board	≤ 40 pieces, as trained: cost −9 %, <i>seeds overlap</i> ; greedy +27 pp	56–83 pieces: cost −21 % → −36 %, greedy +40 → +46 pp (16×16 → 20×20)

READING One pattern, three times. Where a symbol’s training exposure ends — more colours than trained with, a shape never seen, more pieces than ever on one board — the symbol policy pays in cost and loses the greedy rollout, while the relational policy is unaffected, because it never used the names. The shape ID falls short even inside its exposure: shape-type equality has to be computed from a 989-way name, and the policy extracts far less of it than the given relation supplies. Every cost gap widens with board size.

A. Colour

CLAIM With colours as equality relations the policy is exactly invariant to renaming colours, and with more colours than it was trained on it is cheaper at matched budget and far more usable with greedy or single-rollout decoding than one-hot colour — with or without relabelling augmentation.

10 colours at test (training: 2–4); boards 75–80 % covered, deeply scrambled; seed means.

board	input	cost ↓	greedy ↑	×1 ↑	×16 ↑
12×12	relational	102.5	0.773	0.952	0.995
	one-hot	133.1	0.251	0.838	0.993
	one-hot + relabelling	120.8	0.370	0.932	1.000
20×20	relational	489.3	0.553	0.969	1.000
	one-hot	1028.5	0.003	0.331	0.763
	one-hot + relabelling	752.5	0.028	0.815	0.998

The same holds at 6 and 8 colours and at 16×16 (not shown). Renaming the colours of a test board changes 0 of the relational policy’s actions, but 33–46 % of the one-hot policy’s and 42–55 % of the augmented one’s. On training-like boards (12×12, ≤ 8 colours) all three solve ≈ 100 % at ×16, so the comparison is made where there is headroom.

READING A one-hot colour is a name, and a name never seen in training is noise. Augmentation buys approximate invariance (×16 holds up) but not the *use* of equality — greedy and cost stay far behind. Equality relations give exact invariance for free, and the policy exploits it.

B. Shape

CLAIM Shape enters only as relations (same piece / same type / can fill) plus a few closed-form numbers. Against a control that is instead given each cell's shape ID — from which every one of those relations is computable — the relational policy is $\approx 30\%$ cheaper and 44 pp better greedily on training shapes, more so on larger boards; on boards dominated by never-seen shapes it is unchanged, while the ID control loses further ground.

12×12; boards 75–80 % covered, deeply scrambled; seed means. The never-seen-shape board and its control share size, density, scramble depth and piece sizes; only the shape catalogue differs (cost there on the 94 / 153 boards both policies solved).

test boards	input	cost ↓	greedy ↑	×1 ↑	×16 ↑
training shapes, 10 colours	relational	101.7	0.773	0.952	0.995
	shape ID	141.7	0.315	0.854	0.991
never-seen shapes (6 shapes excluded from all training; 72 % of piece cells), 4 colours	relational	122.4	0.275	0.723	0.989
	shape ID	233.9	0.008	0.117	0.459
same-size training shapes (control board), 4 colours	relational	123.7	0.251	0.686	0.985
	shape ID	219.0	0.019	0.167	0.559

READING A never-seen shape is another pattern of the same yes/no answers, so the relational policy has nothing new to learn: its rates on the never-seen-shape board match its control (greedy 0.28 vs 0.25, ×1 0.72 vs 0.69). The ID control holds the same information and more, yet uses far less of it — and an ID that never occurred in training is a slot with no learned meaning, so it drops a further 5 pp at ×1 and 10 pp at ×16. Removing the shape relations outright costs $\approx 40\%$ and 50 pp at 12×12; the ID recovers only part of that. On training shapes it still solves $\approx 99\%$ at ×16: the deficit is cost and commitment, not competence.

C. Piece membership

CLAIM Which cells belong to the same piece enters only as a relation. The control is instead given a per-cell piece ID drawn at random for every board, so it carries exactly the same information — equality of IDs — and nothing else. Up to the trained number of pieces the two forms solve every board at ×16, differ in cost by 9–11 % with seeds overlapping, and differ by 27 pp in greedy solve; beyond it (16×16: 56–61 pieces; 20×20: 79–83) the relational policy is 21–36 % cheaper and 40–46 pp better greedily (means per board size), seeds separated at five of the six board-colour settings and the sixth short by 1 % (table above).

READING A piece ID is a name whose only use is to be compared for equality, and a policy that trained with at most 40 names on a board has never had to compare 80. The relation is that comparison already done — the same computation on any board. On 12×12 the same-piece answer is nearly recoverable from geometry alone, so the forms almost tie there. Board size and piece count grow together in these tests and cannot be separated; what the numbers show is that the membership gap opens only once the piece count passes the trained cap, whereas the colour and shape gaps are open at 12×12 already.

D. Geometry

CLAIM Relative offsets keep the solve ceiling on boards larger than any seen in training; absolute coordinates lose it.

Trained on 12×12, tested on random dense boards (55 % covered). On 12×12 both are fine (greedy 0.978 vs 0.940; ×16 ≥ 0.99); on boards solvable by construction the relational policy holds ×16 = 1.000 out to 32×32.

board	input	greedy ↑	×1 ↑	×16 ↑
16×16	relational	.598 / .566 / .882	0.99	1.000 / 1.000 / 1.000
	absolute	.176 / .160 / .004	0.76	1.000 / 1.000 / .998
20×20	relational	.308 / .170 / .794	0.97	1.000 / 1.000 / 1.000
	absolute	.000 / .012 / .000	0.08	.074 / .022 / .884

READING The absolute code names positions that never occurred in training; the offset code asks the same question at any size. The relational policy's greedy rollout degrades with size, but one sampled rollout recovers 0.97–0.99 — a sharpness effect of deterministic decoding, not lost competence — whereas at 20×20 the absolute policy is not rescued even at ×16 on two of three seeds.